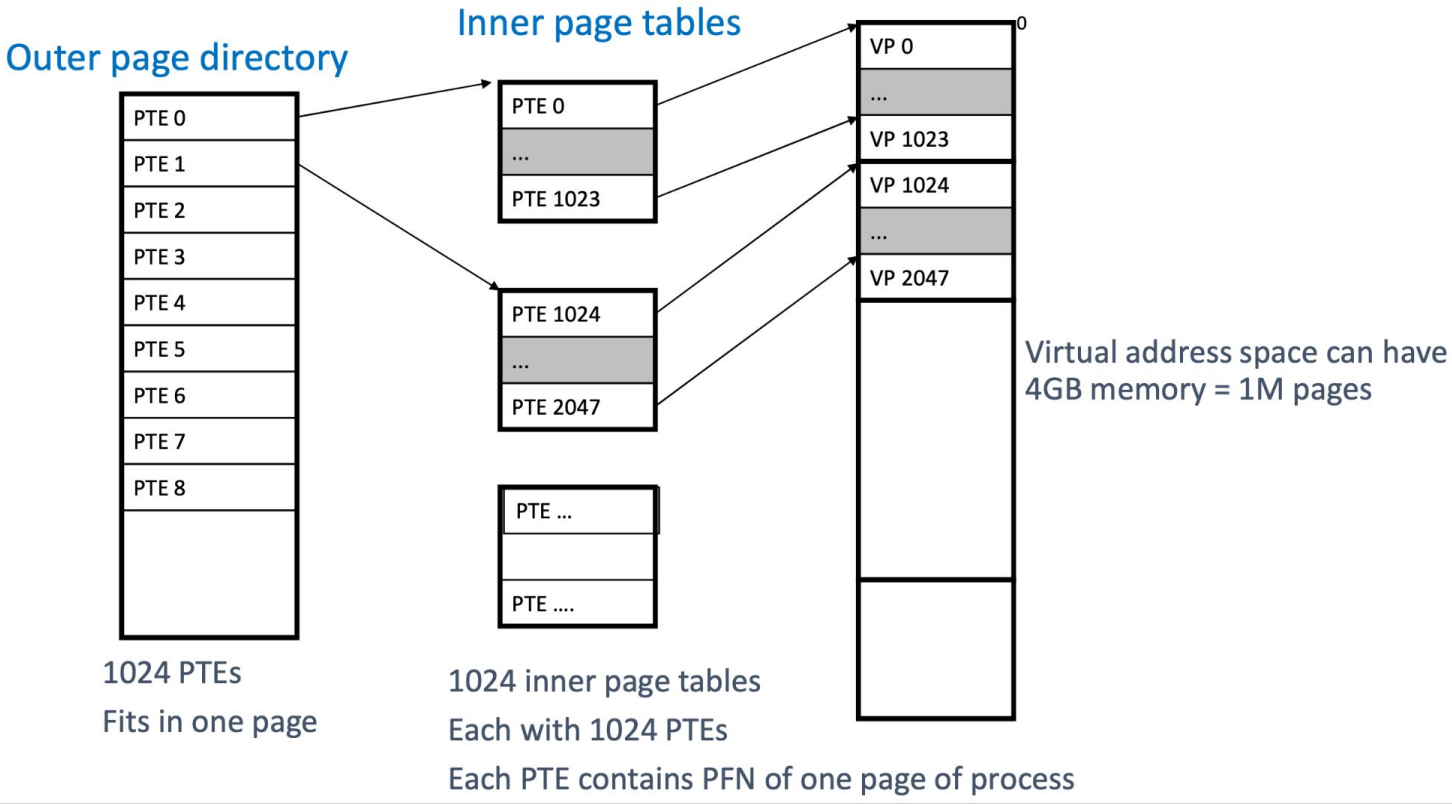
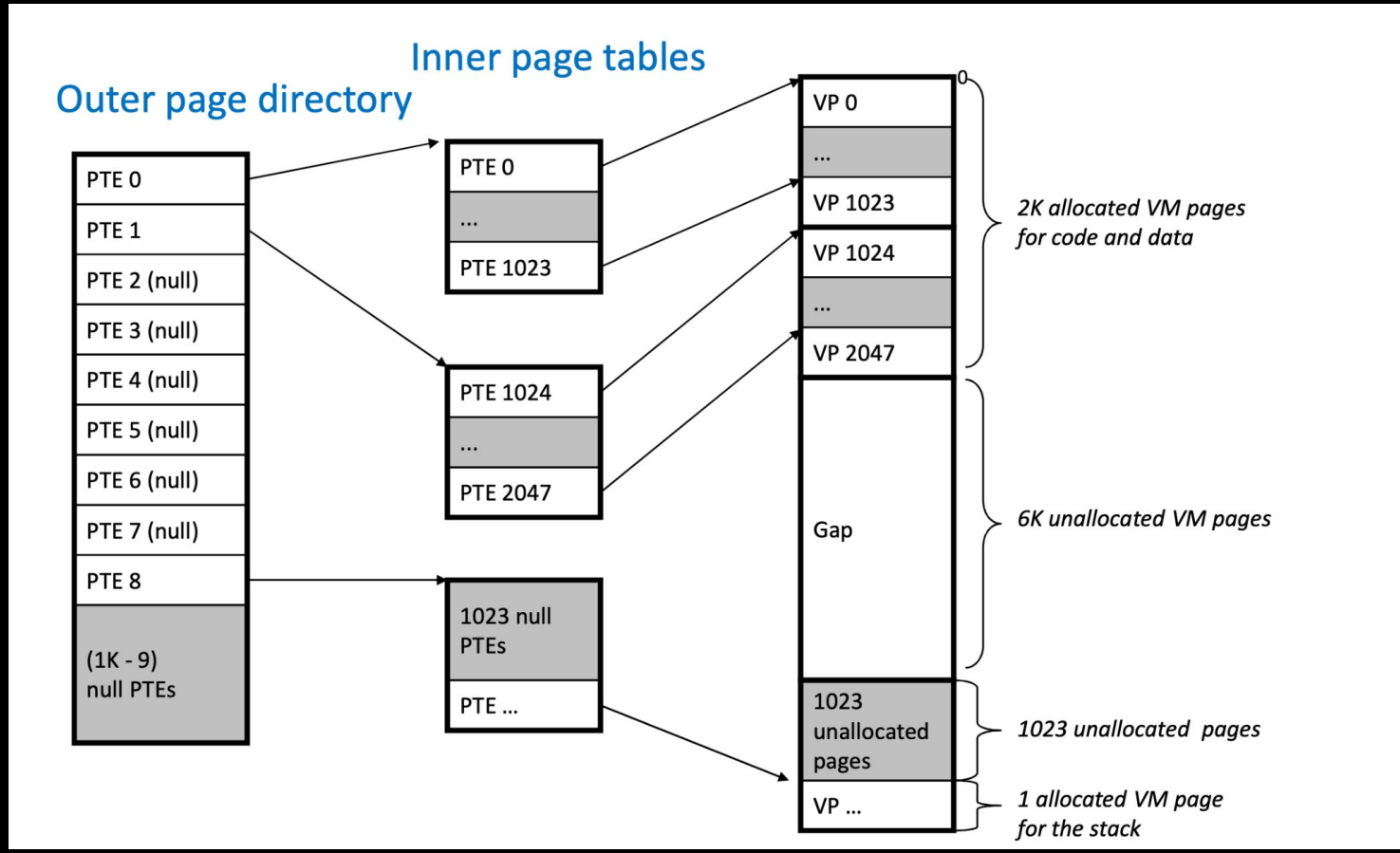
Two-level page table in 32-bit systems

- 4MB page table split into 1024 chunks of 4KB each (to fit in page)
- 1M PTEs split across 1024 pages, each containing 1024 PTEs
- Physical frame numbers of these 1024 chunks stored in an outer page table or page directory
  - *4 byte page table entry each, so outer page directory fits in one page here*
- Page table now has two levels
  - *Outer page table (page directory) has physical frame numbers of 1024 “inner” page table pages*
  - *Each inner page table has physical frame numbers (PTEs) of 1024 pages of the process virtual address space*

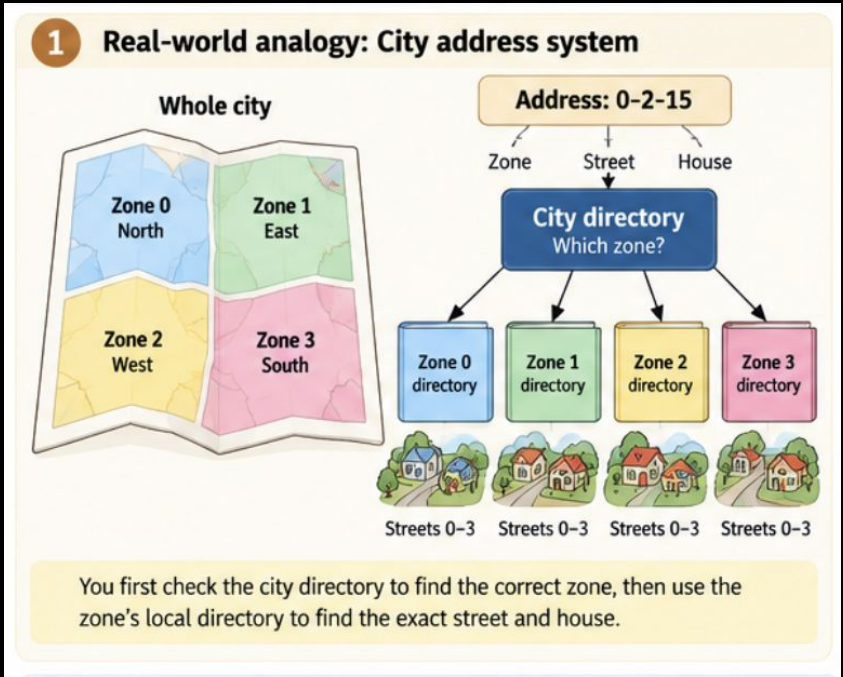

Outer page directory
Inner page tables
PTE 0
PTE 1
PTE 2
PTE 3
PTE 4
PTE 5
PTE 6
PTE 7
PTE 8
PTE 0
...
PTE 1023
PTE 1024
...
PTE 2047
PTE ...
PTE ....
0
VP 0
...
VP 1023
VP 1024
...
VP 2047
Virtual address space can have
4GB memory = 1M pages
1024 PTEs
Fits in one page
1024 inner page tables
Each with 1024 PTEs
Each PTE contains PFN of one page of process

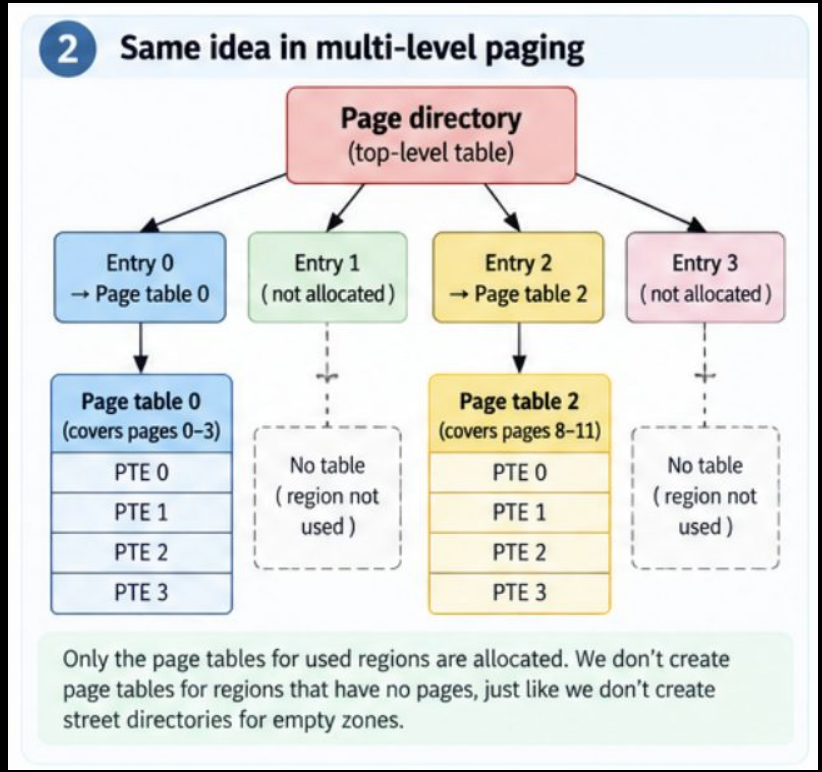
Inner page tables on demand

- Note: not all inner page tables need to be created always, only those with at least one valid entry needed
- Example: Process with 2K pages of code+data, 6K + 1023 unallocated pages in address space, then one page allocated for stack
    - *First two inner page tables are allocated, hold the 2K valid PTEs*
    - *Next 6 inner page tables are not created, the corresponding entries in outer page directory are invalid / null*
    - *In next inner page table, 1023 invalid entries and one valid PTE containing frame number of stack page*
    - *Remaining inner page tables not created, corresponding outer page directory entries are invalid*


Inner page tables
Outer page directory
PTE 0
PTE 1
PTE 2 (null)
PTE 3 (null)
PTE 4 (null)
PTE 5 (null)
PTE 6 (null)
PTE 7 (null)
PTE 8
(1K - 9)
null PTEs
PTE 0
...
PTE 1023
PTE 1024
...
PTE 2047
1023 null
PTEs
PTE ...
0
VP 0
...
VP 1023
VP 1024
...
VP 2047
Gap
1023
unallocated
pages
VP ...
2K allocated VM pages
for code and data
6K unallocated VM pages
1023 unallocated pages
1 allocated VM page
for the stack

Multi-level Paging (City, Zone, Street, House)



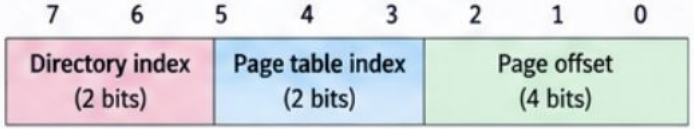


Multi-level Paging

3 Example setup (small numbers)

Virtual address size	:	8 bits (256 bytes)
Page size	:	16 bytes (2^24)
Number of virtual pages	:	16 (0–15)
Two-level paging	:	2 bits + 2 bits + 4 bits

Virtual address (8 bits)



Address format example

Directory	Table	Offset
0 0	1 0	1 0 1 1
(0)	(2)	(11)

4 Actual tables with a simple example

Page directory (4 entries)

Index	Points to
0	Page table 0
1	Not allocated
2	Page table 2
3	Not allocated

Page table 0 (for pages 0–3)

Index	PFN	Valid
0	5	1
1	7	1
2	-	0
3	-	0

Covers virtual pages 0–3 (directory index 0)

Page table 2 (for pages 8–11)

Index	PFN	Valid
0	9	1
1	-	0
2	-	0
3	-	0

Covers virtual pages 8–11 (directory index 2)

Multi-level Paging

5 Walkthrough: translate one address

Translate virtual address: 00101011 (binary)

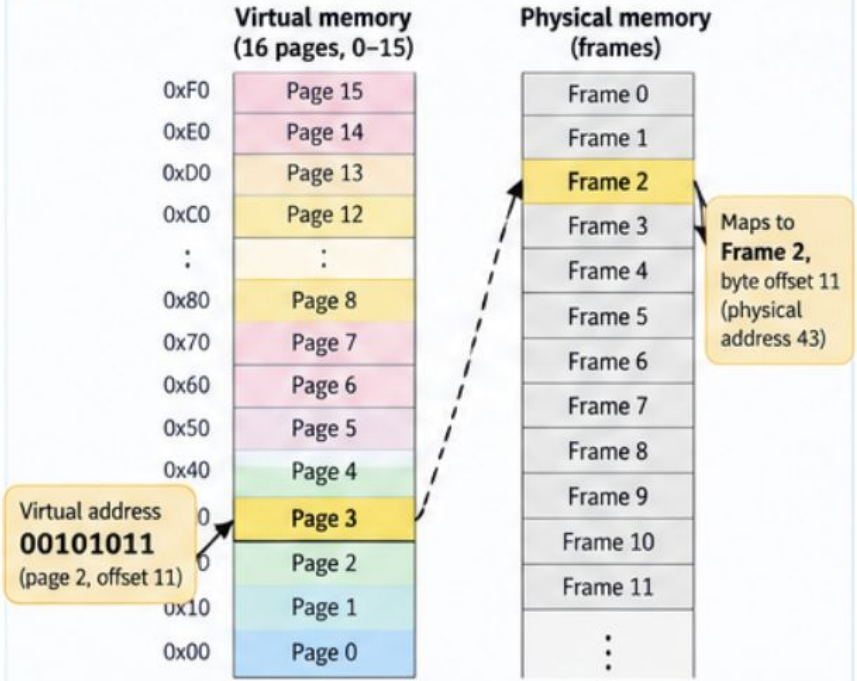
00	10	1011
Directory index (0)	Table index (2)	Offset (11)

1. **Use directory index 00 (0)**
   - Look at page directory entry 0 → points to Page table 0.
2. **Use table index 10 (2)**
   - Look at entry 2 in Page table 0.
   - It maps to physical frame 2 (valid).
3. **Use offset 1011 (11)**
   - Page size = 16 bytes.
   - Frame 2 base address = 2 × 16 = 32.
   - Add offset 11 → 32 + 11 = 43.

Result

Virtual address 00101011 → Physical address **00101011**
(in this example, both happen to be the same: 43 in decimal)

6 Visual view: address to physical memory



Revisit Virtual Address Space

- What should virtual address space/page table of process have? Any memory that the process needs to access during its execution
  - *Its own memory image: code, data, stack, heap*
  - *Other common memory it needs to access: shared language libraries, OS*
- Why? MMU allows access to memory only via virtual addresses
  - *Can only access physical memory mapped in page table at some virtual address*
  - *So all physical memory needed by process should be mapped into address space*
- OS binary image (kernel code, data) is mapped into the virtual address space of every process at addresses not used by process (high VA)
- Why is this done? Easy to jump to OS code during a trap

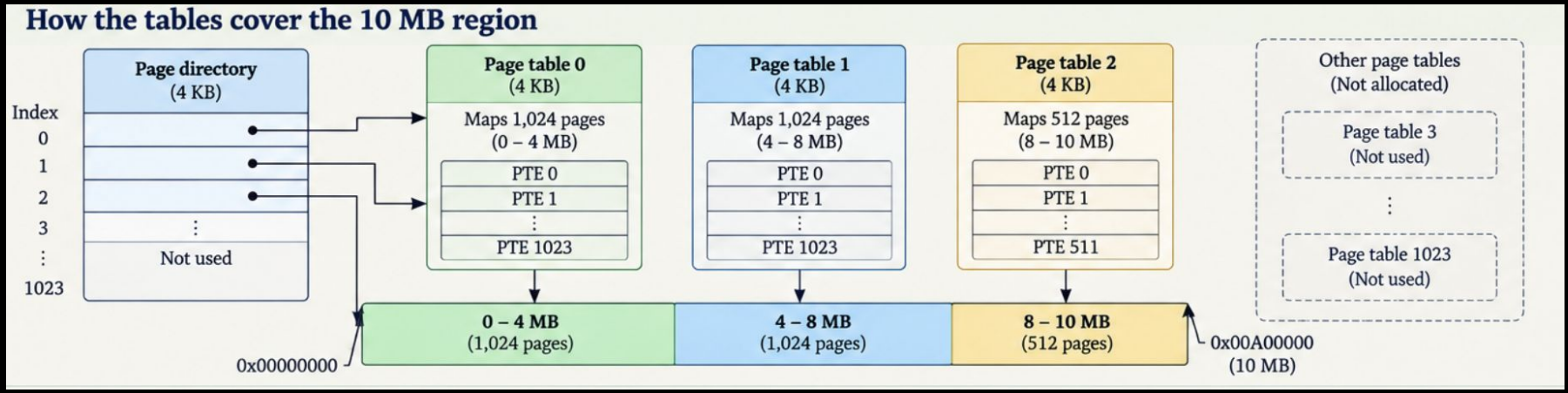
How many tables are required for 10 MB?

How many tables are required for 10 MB?

10 MB requires 3 second-level page tables, plus 1 page directory — 4 tables total.

- Pages used: $10\ \text{MB} \div 4\ \text{KB} = 2,560$
- Entries per second-level table: $1,024$
- Second-level tables: $\lceil 2,560/1,024 \rceil = 3$
- Total page-table memory: $4\ \text{KB} + 3(4\ \text{KB}) = 16\ \text{KB}$
- Memory saved versus a single-level table: $4\ \text{MB} - 16\ \text{KB} \approx 3.984\ \text{MB}$ (about 99.6%).

How many tables are required for 10 MB?



How many tables are required for 10 MB?

5 Memory comparison

Single-level page table

1,048,576 entries × 4 bytes = 4,194,304 bytes

= 4 MB per process



4 MB

Two-level page table (for 10 MB usage)

1 page directory	= 4 KB
3 second-level tables	= 3 × 4 KB = 12 KB

Total = 16 KB



16 KB

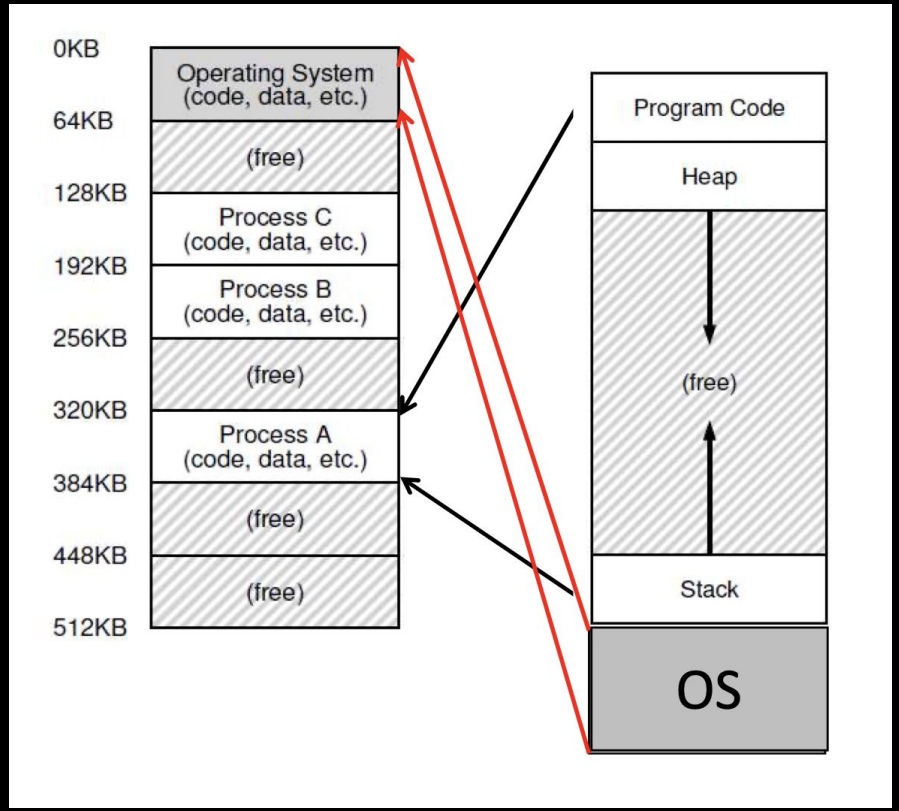
Memory saved

4 MB – 16 KB = 3.984 MB

(≈ **99.6%** less page table memory for this example)

OS is part of address space of every process

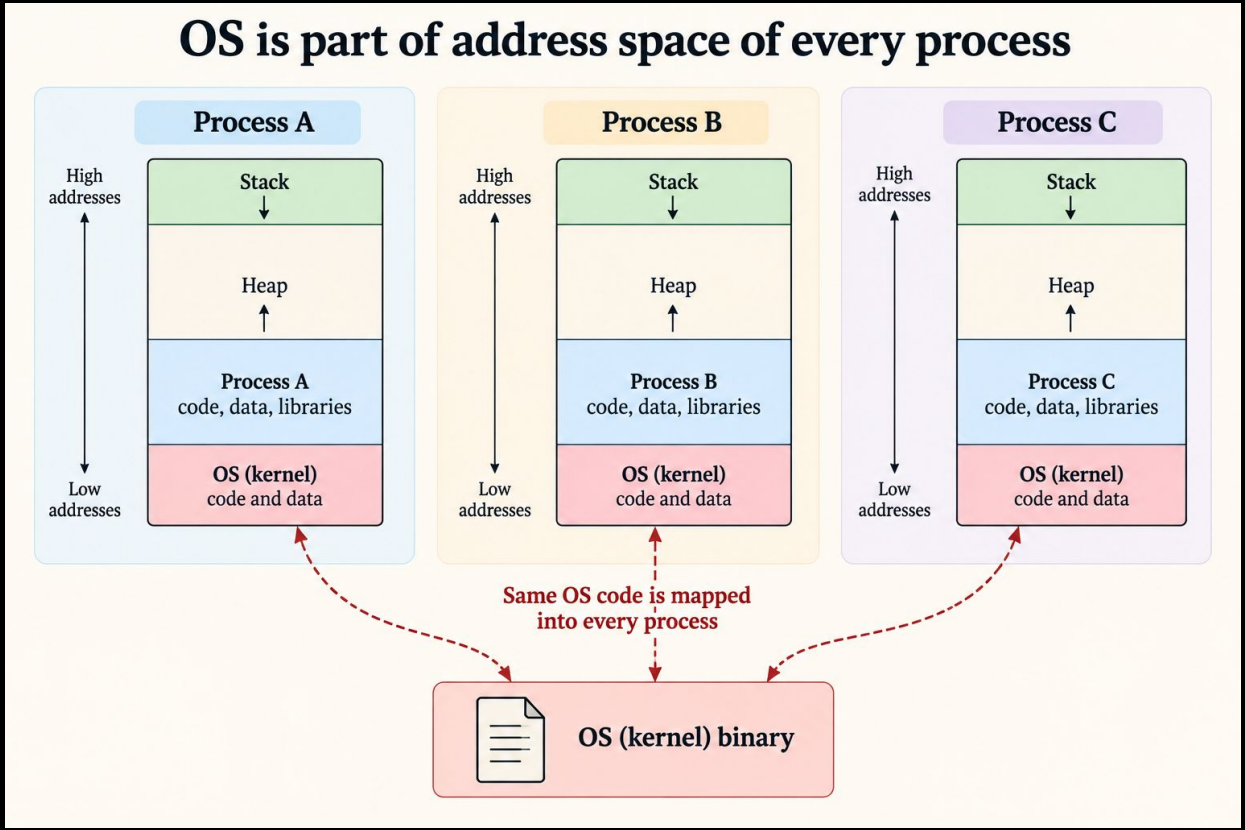
- OS is not a separate process with its own address space
- Instead, OS code is part of the address space of every process
- A process sees OS as part of its code (e.g., like a library)
- During trap, process jumps to high virtual address



OS is part of address space of every process

- OS code/data assigned high virtual addresses
  - *Compiler ensures high virtual addresses not used by user code*
- OS virtual addresses are mapped to physical addresses of OS via page table entries of every process
- There is only one copy of OS code/data in RAM
  - *Loaded into RAM at low physical addresses during system bootup*
- Page tables of all processes map to same OS physical addresses
  - *Same high virtual addresses map to same physical addresses of OS code*

OS is part of address space of every process



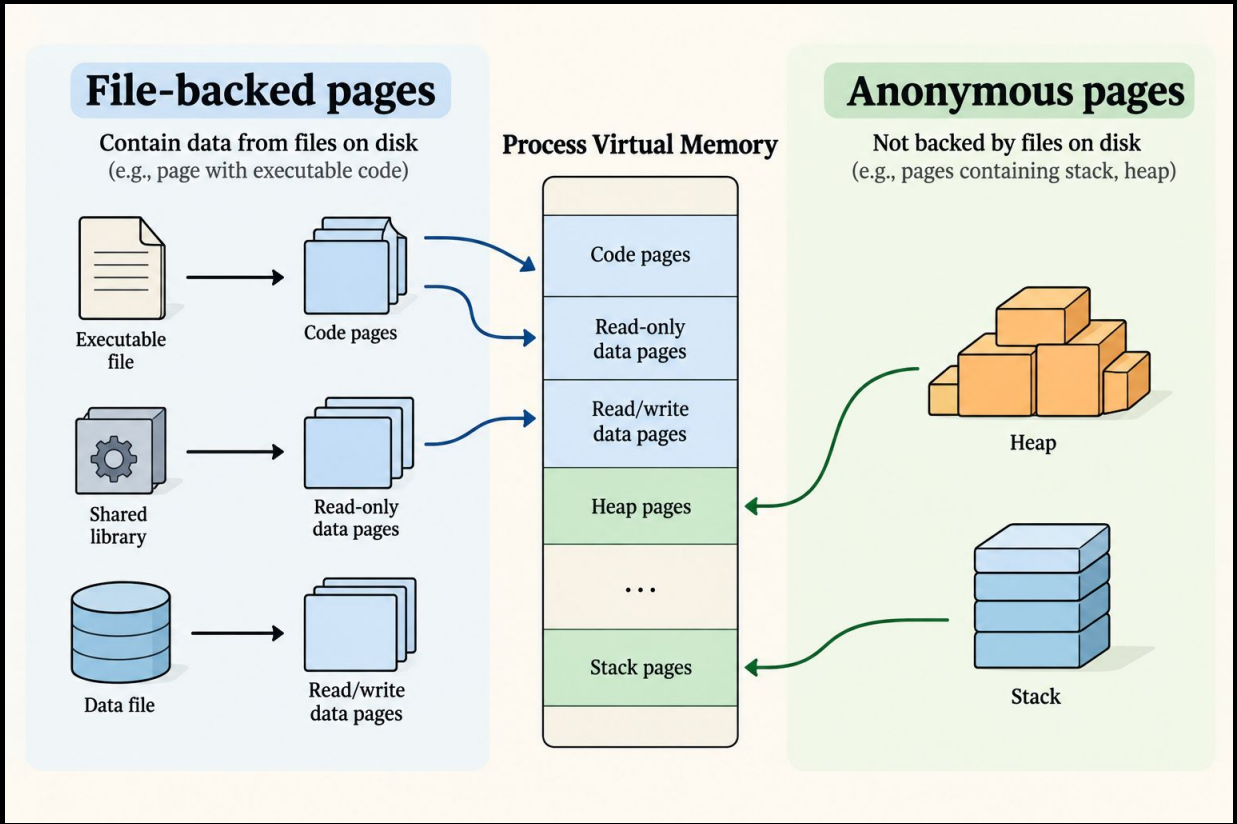
Page-level isolation and security

- How is OS code/data protected from illegal access by user?
- Page table has permissions for every memory page
  - *Whether read/write or read-only (code pages are read-only)*
  - *Whether page can be accessed in user mode or kernel mode*
- Page table mappings for OS code are protected to allow access only when CPU is in kernel mode
  - *CPU in user mode cannot access high virtual addresses of OS code*
  - *CPU in kernel mode (after trap instruction) can access OS code/data*
- MMU traps to OS if any violation detected during memory access, ensures user programs can only access memory they are permitted to access

File backed, anonymous, dirty pages

- Pages in the memory image of a process are of two types
  - *File-backed pages contain data from files on disk (e.g., page with executable code)*
  - *Anonymous pages are not backed by files on disk (e.g., pages containing stack, heap)*
- Further classification into dirty and non-dirty pages
- Dirty pages: pages whose content is different from their copy on disk
  - *E.g., file-backed pages whose contents have changed from file*
  - *E.g., anonymous pages whose contents have changed since last read from swap*
- PTE has information to track these types of pages, including dirty bit

File-backed vs Anonymous Pages



Dirty and Non-dirty pages

